CS405 Project 1 Report

Transformations, Projection & Curves from Paintings

Ömer Mert Özel · Student ID: 31998

1 Painting

Şeker Ahmed Paşa — Kavunlu Natürmort (Sakıp Sabancı Museum; Fig. 1). Chosen for a clear organic silhouette suited to Part D curve tracing, with space in front of the textured quad for Part A objects and the Part B lamp.



Figure 1: Chosen painting: *Kavunlu Natürmort*.

2 Part A — Transformations

Matrices are `Float32Array(16)`, column-major ($p' = Mp$). `multiply(A,B)` returns $A \cdot B$ (B first). Implemented: identity, multiply, translate, rotate $x/y/z$, scale, transpose, `invertRigid` for $M = T \cdot R$ via $M^{-1} = [R^\top \mid -R^\top t]$.

$\geq$3 cubes in front of the painting; one UI-driven (translate, y-rotate, uniform scale). A switch selects $M = T \cdot R \cdot S$ or $M = S \cdot R \cdot T$; both matrices are shown (Figs. 2–3). Scaling after translation also scales the translation, so the centre moves; scaling before does not. Elsewhere I use $M = T \cdot R \cdot S$.

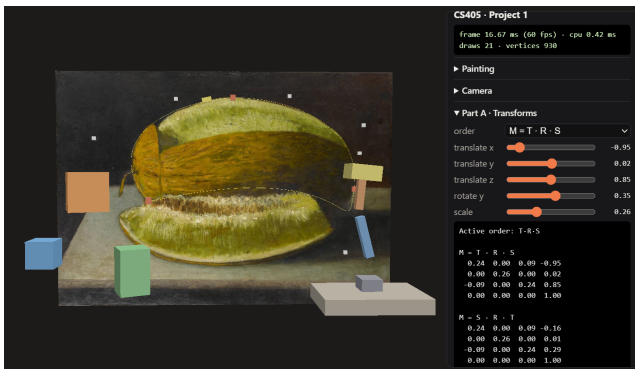


Figure 2: $T \cdot R \cdot S$.

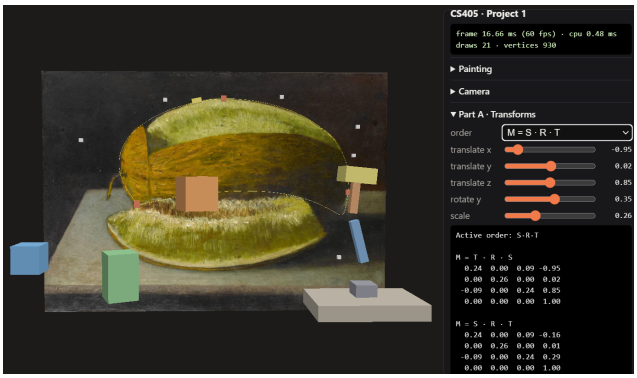


Figure 3: Same T, R, S, order $S \cdot R \cdot T$.

3 Part B — Scene graph

`SceneNode`: local, parent/children, world via world = parent.world · local. Tree (≥3 levels, 5 nodes): root → table → lampBase → arm1 → arm2 → shade. `arm1`/`arm2` animate; debug axes optional.

Hand check. a_1=0.25, a_2=−0.35: M_{table}=$T(0.95,-0.7,0.9)$, M_{base}=$T(0,0.1,0)$, M_{arm1}=$T(0,0.18,0)R_z(a_1)$, M_{arm2}=$T(0,0.32,0$ M_{shade}=$T(0,0.26,0)$. $W = M_{\text{table}}M_{\text{base}}M_{\text{arm1}}M_{\text{arm2}}M_{\text{shade}}$ maps the shade origin to $\approx (0.90, 0.15, 0.90)$, matching $W(0,0,0,1)^\top$.

4 Part C — Camera and projection

View. $f = (t-e)/\|t-e\|$, $w = -f$, $s = (u \times w)/\|u \times w\|$, $v = w \times s$ (if $u \parallel w$, swap up). View columns (s, v, w), translation $(-\langle s,e\rangle, -\langle v,e\rangle, -\langle w,e\rangle)$; looks down −z, +y up.

Projection (WebGPU depth). With $f = 1/\tan(\text{fovY}/2)$,

$$P = \begin{bmatrix} f/\text{aspect} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{\text{far}}{\text{near}-\text{far}} & -1 \\ 0 & 0 & \frac{\text{near far}}{\text{near}-\text{far}} & 0 \end{bmatrix}$$

(near→ 0, far→ 1 after divide). Orthographic uses the same depth range, $w = 1$. Orbit/zoom; panel shows eye, target, fov, near, far. Inspection draws the frustum (Fig. 6). Same pose, both projections: Figs. 4–5. I use near=0.1, far=100: tight enough for the scene; extremes waste depth precision.

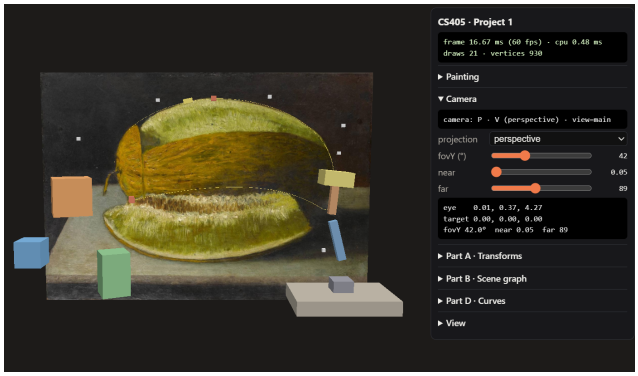


Figure 4: Perspective.



Figure 5: Orthographic (same pose).

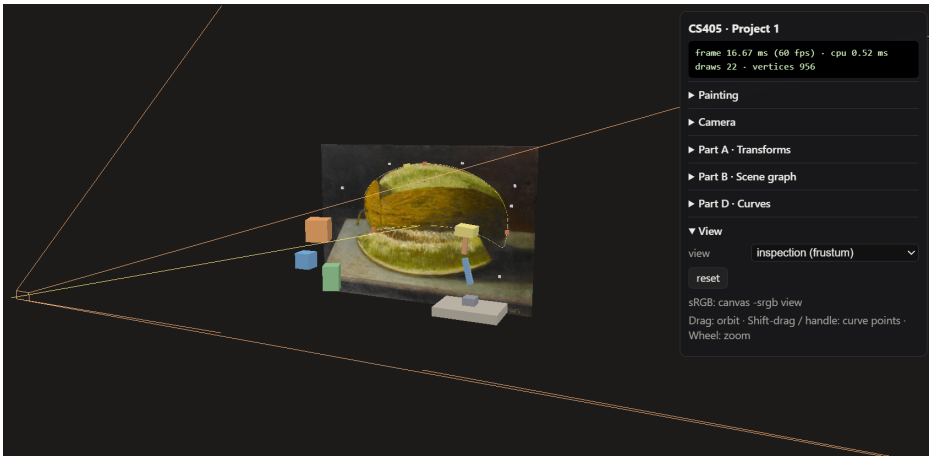


Figure 6: Inspection view with the main camera frustum.

5 Part D — Curves

Bernstein: $B_0 = (1-t)^3$, $B_1 = 3(1-t)^2 t$, $B_2 = 3(1-t)t^2$, $B_3 = t^3$; $B(t) = \sum B_i(t)P_i$. At t=0, B_0=1 $\Rightarrow$ $B(0)$=P_0; at t=1, B_3=1 $\Rightarrow$ $B(1)$=P_3. C^1 at joint K=P_3=Q_0: Q_1=$2K - P_2$ (UI-enforced). Three C^1 segments + Catmull–Rom; drag in painting UV; rider follows $C(t)$ with tangent orientation (Fig. 7). Exported JSON: Appendix 6 (also in the GitHub repo as `curve.json`).

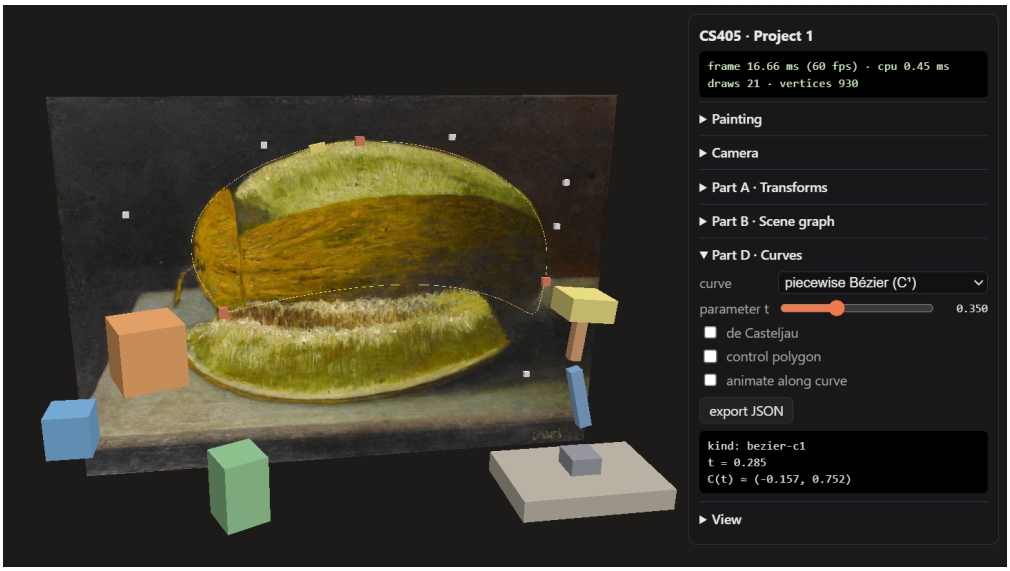


Figure 7: Piecewise C^1 Bézier over the melon silhouette.

6 Integration, performance, issues

One WebGPU app (Parts A–D); modules `math`/`scene`/`render`/`ui`. **NVIDIA GeForce RTX 3050 Laptop GPU**: ∼16.7 ms/frame (∼60 fps), ∼20 draws / $\sim 10^3$ verts. Issues: (1) default UV missed the melon — retuned + re-exported JSON; (2) Part A/B spatial overlap — separated left/right; (3) agent `Co-authored-by` trailers — final commits made manually.

Appendix: curve JSON


```
{
  "painting": "kavunlu",
  "kind": "bezier-c1",
  "segments": [
    [[0.27046, 0.545237], [0.112206, 0.283759], [0.353739, 0.117869], [0.521813, 0.110962]],
    [[0.521813, 0.110962], [0.689887, 0.104055], [0.912779, 0.22637], [0.887181, 0.49961]],
    [[0.887181, 0.49961], [0.861583, 0.77285], [0.901182, 0.345066], [0.272496, 0.551941]]
  ]
}
```